

Bachelorarbeit

**Identification and Interpretation of BMS Signals
Through CAN Bus Reverse Engineering
on the Volkswagen ID. Buzz**

vorgelegt von

Muhamad Alif Izzuwan Bin Ibrahim

Matrikelnummer: 00124399
Studiengang: Elektrotechnik und Elektromobilität (B. Eng.)
Fakultät: Elektrotechnik und Informatik

Erstprüfer: Prof. Dr. rer. nat. Hans-Georg Schweiger
Zweitprüfer: Prof. Dr. Werner Huber
Betreuer: Markus Gregor

Ausgabedatum: 10.03.2026

Abgabedatum: 15.07.2026

Declaration in accordance with §18 (4) No. 7 APO THI

I hereby declare that I have written this thesis independently, that it has not previously been submitted for examination purposes elsewhere, that I have used no sources or aids other than those indicated, and that I have marked verbatim and paraphrased quotations as such.

Place, Date

Ingolstadt, 15.07.2026

Signature

Acknowledgements

The present work was carried out as part of my Bachelor studies in Electrical Engineering and Electric Mobility at Technische Hochschule Ingolstadt.

First and foremost, I would like to thank everyone who supported me during the preparation of this thesis. My sincere thanks go to Prof. Dr. Hans-Georg Schweiger for his weekly supervision and the regular consultation hours that provided invaluable guidance. I also thank Prof. Dr. Werner Huber for agreeing to act as second examiner and for the constructive feedback he provided on the scope of the work.

I am especially grateful to my supervisor Markus Gregor, who accompanied this work intensively and whose deep knowledge of the CARISSMA research environment shaped the experimental setup around the Volkswagen ID. Buzz.

Finally, I would like to thank my parents, who believed in me throughout my studies and supported me both financially and morally during the long weeks of data collection and writing.

Contents

Declaration	ii
Acknowledgements	iii
Contents	v
List of Figures	vi
List of Tables	vii
1. Introduction	I
1.1. Motivation	I
1.2. Problem Statement	2
1.3. Structure of the Thesis	2
2. Fundamentals and State of the Art	4
2.1. Controller Area Network (CAN)	4
2.1.1. Physical and Link Layer	4
2.1.2. Frame Structure	4
2.1.3. Arbitration and Prioritisation	5
2.1.4. Classical CAN vs. CAN-FD	5
2.2. Diagnostics: OBD-II, ISO-TP and UDS	5
2.2.1. The OBD-II Connector (SAE J1962)	6
2.2.2. ISO 15765-2 Transport Protocol	6
2.2.3. Unified Diagnostic Services (UDS)	8
2.2.4. Diagnostic Addressing for Battery Data	8
2.3. The Volkswagen MEB Platform	9
2.4. High-Voltage Battery Systems in BEVs	9
2.4.1. Cell, Module and Pack	9
2.4.2. Battery Management System	9
2.4.3. Signals of Interest	10
2.5. State of the Art of CAN Bus Reverse Engineering	10
3. Materials and Methods	II
3.1. Test Vehicle: Volkswagen ID. Buzz	II
3.2. Hardware Setup	II
3.2.1. CANedge2 Data Logger	II
3.2.2. OBD-II Connection	12
3.3. Software Tool Chain	12
3.4. Why Active UDS Instead of Passive Reverse Engineering	13

3.5.	Reverse-Engineering Methodology	13
3.6.	UDS Diagnostic Procedure for Battery Data	14
4.	Results	16
4.1.	UDS Logging Campaign	16
4.2.	Active Battery Data Retrieval via UDS	16
4.2.1.	Response Coverage	16
4.2.2.	Extracted Battery Parameters	17
4.2.3.	Anomalies	17
4.2.4.	Validation Against the Physical Vehicle	18
5.	Discussion	19
5.1.	Comparison with Related Work	19
5.2.	Limitations	19
5.3.	Practical Applications	19
6.	Conclusion and Outlook	21
6.1.	Summary	21
6.2.	Outlook	21
	Bibliography	23
A.	Appendix	24
A.1.	UDS Request Builder	24
A.2.	Selected UDS Data Identifiers	24

List of Figures

2.1.	Simplified layout of a classical CAN 2.0A data frame. The identifier field is used for bitwise arbitration; the data field carries the payload the application layer is interested in.	4
2.2.	ISO 15765-2 (ISO-TP) protocol data units. Short diagnostic messages fit in a Single Frame; longer responses use First Frame + Consecutive Frames with Flow Control.	7
2.3.	ISO-TP message sequence between the tester (client) and the ECU (server) for a multi-frame UDS response: Single Frame request, First Frame, Flow Control, and Consecutive Frames separated by STmin.	7
3.1.	The CANedge2 data logger (CSS Electronics) used in this thesis. CH1 is connected to the OBD-II HS-CAN bus of the ID. Buzz; CH2 is left in listen-only mode.	11
3.2.	Overall measurement setup. The CANedge2 logger is connected directly to the OBD-II connector of the vehicle, is powered from pin 16 of that connector, and writes MDF4 files to its SD card for offline analysis on the PC.	12

List of Tables

2.1.	Mapping of the diagnostic protocol stack used in this thesis onto the OSI reference model. .	6
2.2.	Key pins of the SAE J1962 OBD-II connector used in this thesis.	6
2.3.	UDS services relevant to the retrieval of battery data.	8
2.4.	Battery parameters targeted by the UDS queries in this thesis.	10
3.1.	Normalised frame format used across the analysis pipeline.	12
4.1.	UDS logging sessions recorded on the VW ID. Buzz (device 2A73E1CC). Frame counts include both the transmitted requests and the logged responses.	16
4.2.	Battery parameters extracted from the VW ID. Buzz via UDS <code>ReadDataByIdentifier</code> (log 00000039). Values are the decoded physical quantities after applying the per-DID scaling of the master PID list.	17
A.1.	The 59 UDS Data Identifiers selected for extraction from the VW ID. Buzz (the rows marked True in the master VW MEB UDS <code>PIDs list.csv</code>). The CANedge2 transmit list is capped at 64 entries, so the selection had to stay below this limit.	24

1. Introduction

This chapter gives an overview of the topic, the motivation, the problem statement and the structure of the thesis.

1.1. Motivation

In recent years, electric mobility has undergone an enormous change. Almost every established automotive manufacturer has announced a portfolio of battery-electric vehicles (BEVs), and governmental policy in the European Union has aligned fleet CO₂ targets with an electrified future [1]. Within these vehicles, the high-voltage traction battery is the single most expensive and safety-critical subsystem. Its operating state – state of charge (SOC), state of health (SOH), pack voltage, pack current, cell temperatures and cell balancing status – determines both the available driving range and the long-term reliability of the vehicle [2, 3].

All of this information is exchanged inside the vehicle over the Controller Area Network (CAN) [4]. However, the manufacturer-specific signal databases (DBC files) that describe how raw CAN frames map to physical quantities are proprietary and not publicly available. For independent researchers, safety labs such as CARISSMA and for fleet-operators who want to monitor their vehicles, this closed ecosystem is a significant obstacle: without signal definitions the raw CAN data is merely a stream of hexadecimal bytes without semantic meaning.

In principle, two approaches can recover this semantic layer. The first is *passive reverse engineering*: tapping the raw battery CAN bus directly, recording a large amount of traffic and using statistical techniques to deduce what each byte represents. This approach, however, requires physical access to the internal battery-management bus on which the BMS broadcasts its raw frames. On the Volkswagen MEB platform that bus is *not* exposed on the OBD-II connector: a central gateway ECU isolates the private vehicle buses, and for security reasons it does not forward the raw BMS broadcast frames to the externally accessible diagnostic bus. A passive byte-level reverse engineering of the BMS signals is therefore not feasible from the OBD-II port without invasively tapping the high-voltage battery wiring of the vehicle.

The second approach – and the one pursued in this thesis – is *active diagnostic reverse engineering*: using the standardised Unified Diagnostic Services (UDS) protocol [5] transported over ISO 15765-2 (ISO-TP) [6] via the on-board diagnostic (OBD-II) connector [7] to request specific Data Identifiers (DIDs) from the battery management ECU through the gateway. The gateway *does* route addressed UDS diagnostic requests to the BMS, so the data becomes reachable. This is still a reverse-engineering task because the meaning, byte layout and scaling of these manufacturer-specific DIDs are undocumented and proprietary: each positive response is a stream of raw bytes whose physical interpretation must be recovered and then **validated by comparing the decoded value directly against the physical readings of the test vehicle** (dashboard SOC, measured pack voltage, ambient temperature, and so on).

The present work applies this approach to a Volkswagen ID. Buzz built on the MEB platform. The ID. Buzz represents Volkswagen's state-of-the-art modular electric architecture, which makes it a particularly interesting test object: its electric and electronic architecture is representative of the next decade of mass-market BEVs from the Volkswagen Group.

1.2. Problem Statement

The main goal of this thesis is to reverse engineer the high-voltage battery signals of a Volkswagen ID. Buzz through its OBD-II diagnostic port. Because the raw BMS broadcast bus is isolated by the gateway, the work focuses on actively requesting battery Data Identifiers via the UDS protocol and recovering their physical meaning. The primary deliverable is the ability to request and decode the high-voltage battery parameters (in particular state of charge, pack voltage, cell voltages and cell-temperature statistics) and to confirm them against the physical state of the vehicle.

Within this overall objective the work addresses the following research questions:

1. Which CAN bus is exposed on the OBD-II connector of the Volkswagen ID. Buzz, and why are the raw BMS broadcast frames not directly observable on it?
2. Which UDS services and Data Identifiers of the battery management ECU are reachable through the gateway from the OBD-II connector without a security-access unlock?
3. How can the byte layout and scaling of these undocumented, manufacturer-specific DIDs be reverse engineered into physical battery parameters?
4. How well do the decoded UDS values agree with the physical reference values of the test vehicle (dashboard SOC, measured pack voltage, ambient temperature), and where does the method reach its limits?

1.3. Structure of the Thesis

This thesis is divided into six main chapters.

Chapter 1 gives an overview of the task and the structure of the thesis.

Chapter 2 presents the theoretical background by means of a literature review. It covers the CAN bus protocol, the OBD-II connector, the ISO-TP transport protocol, the UDS application layer, the Volkswagen MEB platform, traction-battery management systems in BEVs and the state of the art of CAN bus reverse engineering.

Chapter 3 describes the materials and methods used in this work. The test vehicle, the CANedge2 data logger, the OBD-II connection and the software tool chain based on `asammdf` and Python are documented, together with the UDS-based reverse-engineering methodology and the reasoning for using active diagnostics rather than passive bus tapping.

Chapter 4 presents the results: the UDS logging campaign, the response coverage, and the battery parameters decoded from the BMS and validated against the physical state of the vehicle.

Chapter 5 discusses the findings. The limitations of the approach and the comparison with related work are analysed, and practical applications are outlined.

Chapter 6 concludes the work and gives an outlook on future research directions.

2. Fundamentals and State of the Art

In this chapter the essential fundamentals for understanding this work are explained. The introduction serves to describe the Controller Area Network, the layered diagnostic protocol stack built on top of it (ISO-TP and UDS), the OBD-II interface, the Volkswagen MEB platform and the battery management subsystem of a modern BEV. The chapter closes with a short review of prior art in automotive CAN reverse engineering.

2.1. Controller Area Network (CAN)

The Controller Area Network was originally developed by Bosch in the 1980s and is today standardised in ISO 11898 [4]. It is a serial, differential, multi-master broadcast bus that has become the backbone of automotive embedded networking. Its key properties – bit-wise, non-destructive arbitration, short message length, high reliability through CRC protection and low hardware cost – make it extremely well suited to the distributed control problem of a modern vehicle [8, 9].

2.1.1. Physical and Link Layer

On the physical layer, CAN uses a twisted differential pair (CAN-H and CAN-L) terminated with $120\ \Omega$ resistors at both ends. The bus is driven by each node through a transceiver that produces two complementary states: a *dominant* state (logical 0) and a *recessive* state (logical 1). A dominant bit always overrides a recessive one, which is the key mechanism for the arbitration scheme described below. For in-vehicle networks, bit rates of 500 kbit/s (high-speed CAN for powertrain) and 125 kbit/s (comfort CAN) are the most common values in the Volkswagen Group.

2.1.2. Frame Structure

Every CAN message is transmitted as a frame with the structure shown in Figure 2.1. Apart from the start-of-frame bit, an 11-bit arbitration identifier (or 29-bit extended identifier), and control bits, the frame carries a payload of up to 8 bytes (Classical CAN; CAN-FD supports up to 64 bytes) protected by a 15-bit CRC.

SOF	Identifier	RTR	Control	Data	CRC	ACK	EOF
1	11 bit	1	6 bit	0–8 byte	16 bit	2 bit	7

Figure 2.1.: Simplified layout of a classical CAN 2.0A data frame. The identifier field is used for bitwise arbitration; the data field carries the payload the application layer is interested in.

2.1.3. Arbitration and Prioritisation

CAN uses Carrier-Sense Multiple Access with Collision Resolution (CSMA/CR). When two or more nodes start transmitting simultaneously, each node reads back the bus level during its own transmission. Because a dominant bit (0) always wins over a recessive bit (1), the node with the numerically lower identifier automatically continues to transmit while the others switch to the listen state. This guarantees that the highest-priority message is never delayed by a collision. In an electric vehicle, safety-critical messages (such as the motor torque request or the battery current measurement) are therefore assigned low arbitration IDs, while convenience signals use higher IDs.

2.1.4. Classical CAN vs. CAN-FD

Two variants of the protocol are relevant. *Classical CAN* (CAN 2.0, ISO 11898-1:2003) is limited to 8 payload bytes per frame and a single bit rate for the whole frame. *CAN-FD* (CAN with Flexible Data-rate, standardised in ISO 11898-1:2015) extends this in two ways: the payload grows to up to 64 bytes, and the data phase of the frame may be transmitted at a higher bit rate than the arbitration phase, while the arbitration mechanism itself is unchanged.

The diagnostic CAN bus exposed on the OBD-II connector of the ID. Buzz operates as a **Classical CAN bus at 500 kbit/s with 11-bit broadcast identifiers**; the diagnostic exchange with the battery ECU additionally uses 29-bit extended identifiers (Section 2.2.4). It is *not* CAN-FD. This is the decisive constraint behind the methodology of this thesis: because each frame carries at most 8 bytes, every UDS response longer than 7 payload bytes must be segmented by ISO-TP, and a logger that can only emit single CAN frames cannot read those multi-frame identifiers (Section 3.6). The data logger used here (CANedge2) electrically supports both Classical CAN and CAN-FD, but is configured for Classical CAN at 500 kbit/s to match the vehicle, with the CAN-FD data bit rate left defined (2 Mbit/s) only so that the controller can initialise.

2.2. Diagnostics: OBD-II, ISO-TP and UDS

While the CAN bus is the low-level transport medium, automotive diagnostics rely on two additional layers built on top of CAN: a transport protocol that allows messages longer than 8 bytes to be fragmented and reassembled (ISO-TP), and an application layer that defines services, sub-functions and data identifiers (UDS). Both are accessed from outside the vehicle via a standardised connector: the OBD-II port.

It is useful to locate each of these protocols in the OSI reference model before going into detail, because the rest of the thesis repeatedly refers to “which layer” a given problem belongs to. The diagnostic stack used here maps onto the OSI layers as summarised in Table 2.1: CAN occupies the two lowest layers, ISO-TP provides the network/transport function that classical CAN itself lacks, and UDS is the application protocol that the battery management ECU actually speaks.

Table 2.1.: Mapping of the diagnostic protocol stack used in this thesis onto the OSI reference model.

OSI layer	Protocol	Role in this work
7 – Application	UDS (ISO 14229)	Services, sub-functions and Data Identifiers; 0x22 <i>ReadDataByIdentifier</i> returns the battery parameters.
6/5	(not used)	No presentation/session layer in CAN diagnostics.
4/3 – Transport/Network	ISO-TP (ISO 15765-2)	Segments UDS messages longer than 7 bytes into First/Consecutive frames and reassembles them.
2 – Data link	CAN (ISO 11898-1)	Frame format, arbitration, CRC, acknowledgement.
1 – Physical	CAN (ISO 11898-2)	Differential CAN-H/CAN-L pair, 500 kbit/s, 120 Ω termination.

2.2.1. The OBD-II Connector (SAE J1962)

All passenger cars sold in the European Union since 2004 (diesel: 2006) must expose a 16-pin diagnostic connector in the driver's footwell area, standardised by SAE J1962 [7]. Two pins carry the high-speed CAN bus used for emissions diagnostics (pin 6 = CAN-H, pin 14 = CAN-L). On many manufacturers' vehicles, additional pins are used by a *gateway ECU* that multiplexes private manufacturer buses onto the OBD-II connector only for authenticated diagnostic tester sessions.

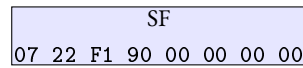
Table 2.2.: Key pins of the SAE J1962 OBD-II connector used in this thesis.

Pin	Signal	Function
4	Chassis ground	Vehicle chassis reference.
5	Signal ground	Signal return.
6	CAN-H (HS-CAN)	High-speed CAN 500 kbit/s, used for OBD-II and UDS.
14	CAN-L (HS-CAN)	Complementary line to pin 6.
16	Battery positive	Permanent +12 V from the vehicle battery (used by the data logger).

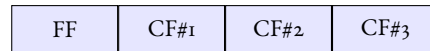
2.2.2. ISO 15765-2 Transport Protocol

A single classical CAN frame can carry only up to 8 bytes. UDS responses for an identifier such as “read VIN” or “read DID” routinely need more than 8 bytes. The ISO 15765-2 transport protocol [6] solves this by fragmenting the application-layer PDU into multiple CAN frames of four possible types: *Single Frame* (SF)

for up to 7 bytes of payload, *First Frame* (FF) plus a sequence of *Consecutive Frames* (CF) for longer messages, and *Flow Control* (FC) frames for sender throttling.



Single Frame: up to 7 payload bytes.



Multi-frame: First Frame + Consecutive Frames, flow-controlled by the receiver.

Figure 2.2.: ISO 15765-2 (ISO-TP) protocol data units. Short diagnostic messages fit in a Single Frame; longer responses use First Frame + Consecutive Frames with Flow Control.

The temporal interplay of these frame types between the diagnostic client (tester) and the server (ECU) is shown in Figure 2.3. The tester opens the exchange with a Single Frame request; the ECU answers with a First Frame, waits for the tester's Flow Control frame and then streams the Consecutive Frames, spaced by the minimum separation time ST_{min} requested in the Flow Control frame.

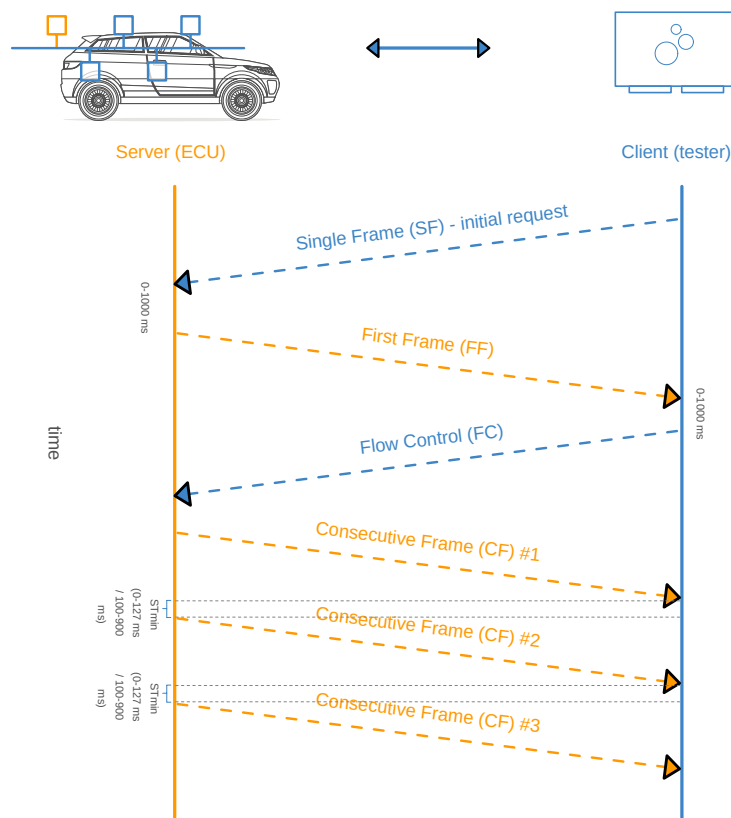


Figure 2.3.: ISO-TP message sequence between the tester (client) and the ECU (server) for a multi-frame UDS response: Single Frame request, First Frame, Flow Control, and Consecutive Frames separated by ST_{min} .

2.2.3. Unified Diagnostic Services (UDS)

UDS, standardised in ISO 14229 [5], is the **application-layer (OSI layer 7)** protocol used for all modern vehicle diagnostics, ECU programming and parameter reading. It sits at the very top of the diagnostic stack of Table 2.1: it defines *what* is requested (services, sub-functions, Data Identifiers), while it deliberately says nothing about *how* the bytes reach the ECU. The segmentation of long messages is delegated to ISO-TP one layer below (transport/network), and the actual bit transport to CAN at the data-link and physical layers. The same UDS service set therefore runs unchanged over CAN, CAN-FD, FlexRay, K-Line or DoIP/Ethernet – only the lower layers differ.

Each UDS request is a byte stream starting with a one-byte Service Identifier (SID). The response either re-uses the SID increased by 0x40 (positive response) or uses 0x7F followed by the original SID and an NRC (negative response code). For example, a read request with SID 0x22 is answered positively with SID 0x62, or negatively with 0x7F 22 NRC – where the NRC 0x31 means *requestOutOfRange* (unknown DID/ECU) and 0x33 means *securityAccessDenied*.

The services most relevant to this thesis are summarised in Table 2.3. In particular, service 0x22 (*ReadDataByIdentifier*) allows a tester to request the value of a specific two-byte Data Identifier (DID). Manufacturer-specific DIDs in the range 0xF100–0xFFFF are routinely used by Volkswagen to expose battery parameters.

Table 2.3.: UDS services relevant to the retrieval of battery data.

SID	Service	Purpose
0x10	DiagnosticSessionControl	Switch between default, programming or extended session.
0x22	ReadDataByIdentifier	Read the value stored under a 16-bit DID.
0x27	SecurityAccess	Unlock protected data ranges via seed/key.
0x2E	WriteDataByIdentifier	Write a value under a DID (not used here).
0x3E	TesterPresent	Keep an extended session alive.

2.2.4. Diagnostic Addressing for Battery Data

For a VW Group vehicle built on the MEB platform, the battery management ECU (“HV battery”, BMS) answers on its own diagnostic CAN ID pair using 29-bit extended addressing. On the ID. Buzz examined here the BMS is reached with request ID 0x17FC007B and replies on 0x17FE007B. Other ECUs follow the same 0x17FC00xx/0x17FE00xx scheme (e.g. DC/DC converter 0x17FC00B9/0x17FE00B9, front EDU 0x17FC0076/0x17FE0076), while a few comfort/gateway ECUs answer on classic 11-bit pairs (e.g. climate 0x746/0x7B0, gateway 0x710/0x77A). A functional request to 0x7DF (the OBD-II broadcast request ID) reaches all emission-relevant ECUs simultaneously and is the starting point for any tester that does not yet know the physical addressing.

2.3. The Volkswagen MEB Platform

The Modular Electric Drive Matrix (*Modularer E-Antriebs-Baukasten*, MEB) is Volkswagen Group's dedicated BEV platform, first used in production on the ID.3 in 2019 [10, 1]. All ID-series vehicles – including the ID. Buzz used in this thesis – share the same underlying architecture: a modular skateboard chassis with a flat battery pack between the axles, a rear-mounted permanent-magnet synchronous motor (single-motor variants) or an additional asynchronous motor on the front axle (4Motion variants), and a fully redesigned in-vehicle electrical and electronic architecture.

From an electronic networking point of view, the MEB is organised around a small number of domain controllers (drive, chassis, comfort, infotainment, charging) connected to a central gateway ECU. The gateway isolates the private manufacturer CAN buses from the OBD-II connector; external testers see either the OBD-II emission CAN bus directly or – in extended diagnostic sessions – a subset of the internal buses routed through the gateway by UDS.

The traction battery of the ID. Buzz (Pro variant) has a gross capacity of 82 kWh (77 kWh net) at a nominal pack voltage of around 400 V. It is managed by a dedicated Battery Management Controller (BMC) and a series of cell-supervision circuits (CSC) that continuously report cell voltages and temperatures to the BMC over an internal battery CAN.

2.4. High-Voltage Battery Systems in BEVs

The lithium-ion pack of a modern BEV is not simply a larger version of a 12 V starter battery. It is an active system composed of several hundred individual cells, grouped into modules, managed by a dedicated electronic hierarchy.

2.4.1. Cell, Module and Pack

A single Li-ion cell has a nominal voltage of 3.6–3.7 V. In the ID. Buzz, cells are connected in series to reach the pack nominal voltage, and in parallel to reach the desired capacity. The cells are organised into modules; each module has its own cell-supervision circuit that measures individual cell voltages (one measurement per cell) and one or more module temperatures.

2.4.2. Battery Management System

The Battery Management System (BMS) is responsible for keeping the pack within its safe operating area. Its core duties are:

- monitoring every cell voltage and every module temperature;
- estimating the pack state of charge (SOC) and state of health (SOH);
- controlling the main positive and negative contactors;

- performing cell balancing (passive or active) during charging;
- reporting power limits (charge/discharge) to the inverter and the charger.

SOC in particular is not a directly measurable quantity. It is estimated by the BMS from a combination of coulomb counting, open-circuit-voltage look-ups and model-based filters such as Kalman filters [11, 2, 3]. The estimate that the BMS reports to the driver’s dashboard is therefore a conservative, smoothed version of a much richer internal state.

2.4.3. Signals of Interest

For this thesis, the following battery parameters are considered the primary targets:

Table 2.4.: Battery parameters targeted by the UDS queries in this thesis.

Parameter	Typical scale	Expected DID range
Pack state of charge	0–100 %, 0.5 % resolution	manufacturer-specific
Pack terminal voltage	0–500 V, 0.1 V resolution	manufacturer-specific
Pack current	±500 A, signed, 0.1 A res.	manufacturer-specific
Cell voltage min / max	2.5–4.25 V, 1 mV resolution	manufacturer-specific
Cell temperature min / max	−40 °C–+80 °C	manufacturer-specific
State of health	0–100 %	manufacturer-specific

2.5. State of the Art of CAN Bus Reverse Engineering

The systematic reverse engineering of automotive CAN networks has a long history in the security and the research community. Koscher et al. [12] and Checkoway et al. [13] were among the first to document automated CAN fuzzing and signal identification on production vehicles. Miller and Valasek’s widely-publicised remote exploitation of a Jeep Cherokee [14] showed that unauthenticated CAN networks are a direct attack surface on vehicle safety.

On the open-source side, the `opendbc` [15] repository maintained by comma.ai collects community-contributed DBC files for dozens of vehicles. SavvyCAN [16] is a mature GUI tool for interactive frame-by-frame reverse engineering. Academic work such as Longari et al. [17] uses payload-based anomaly detection to separate normal from injected CAN traffic, which requires a baseline understanding of what each signal should look like – the exact output of a reverse engineering pass.

For electric vehicles in particular, Huybrechts et al. [18] have demonstrated the reverse engineering of battery-related signals on a Tesla Model S, and Pfeffer and Held [19] describe the use of standard diagnostic protocols for battery monitoring. The combination of passive recording and active UDS querying employed in this thesis closely mirrors their methodology.

3. Materials and Methods

This chapter explains the selection of the test vehicle, the preparation of the measurement hardware, the experimental setup and the software tool chain used to record and analyse the CAN data.

3.1. Test Vehicle: Volkswagen ID. Buzz

All measurements were performed on a 2024 Volkswagen ID. Buzz Pro on loan to the CARISSMA research institute. The vehicle is representative of the MEB platform: 77 kWh net battery capacity, rear-wheel drive with a 150 kW permanent-magnet synchronous motor, and the current generation of Volkswagen’s ID software (ID. Software 4.x). Before every measurement session the vehicle was charged to approximately 90 % SOC at the CARISSMA AC wall-box in order to remove any low-SOC power derating from the results.

3.2. Hardware Setup

3.2.1. CANedge2 Data Logger

The CAN frames were recorded with a CANedge2 data logger (CSS Electronics) [20]. This device has two independent CAN channels, each with a 24-bit hardware timestamp, supports CAN Classical and CAN-FD, and stores data on an SD card in the ASAM MDF 4.11 format [21]. Its power is drawn from pin 16 (battery +) of the OBD-II connector, which means the logger stays active as long as the vehicle has power – including the short wake-up interval after the vehicle is unlocked. Channel 1 was configured for Classical CAN at 500 kbit/s; channel 2 was left in listen-only mode for an optional second tap.



(a) Front view: the two DB9 connectors CH1 and CH2.

(b) Rear view with the SD-card slot.

Figure 3.1: The CANedge2 data logger (CSS Electronics) used in this thesis. CH1 is connected to the OBD-II HS-CAN bus of the ID. Buzz; CH2 is left in listen-only mode.

Crucially, the CANedge2 is not only a passive logger: each CAN channel has a configurable *transmit list* that can periodically send up to 64 user-defined CAN frames onto the bus and log the responses in the same MF4 file. In this thesis that transmit list is used to issue the UDS requests directly from the logger, so that **the active UDS retrieval of battery data is performed by the CANedge2 itself** rather than by a separate diagnostic tester. This makes the active and passive data appear in one synchronised, hardware-timestamped recording. The device used here runs firmware 01.08.01 with configuration schema 01.08.

3.2.2. OBD-II Connection

The CANedge2 was connected directly to the vehicle's OBD-II socket with a standard OBD-II-to-DB9 adapter cable on channel 1; no Y-splitter or parallel tester was used. The cable carries the CAN-H, CAN-L, ground and battery-positive lines on pins 6, 14, 4/5 and 16 respectively (compare Table 2.2), so the logger is both powered and bus-connected through the single OBD-II connector.

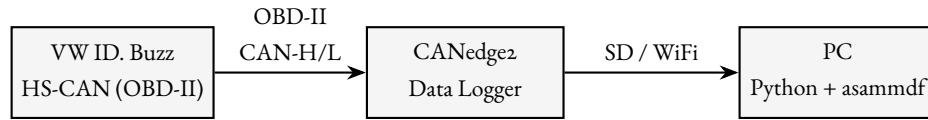


Figure 3.2.: Overall measurement setup. The CANedge2 logger is connected directly to the OBD-II connector of the vehicle, is powered from pin 16 of that connector, and writes MDF₄ files to its SD card for offline analysis on the PC.

3.3. Software Tool Chain

The recorded MDF₄ files were processed entirely in Python 3.13. The actual conversion from the binary MF₄ log into engineering data is performed by the `asammdf` library [21]: it parses the ASAM MDF 4.11 container, decodes the embedded CAN frame groups (arbitration ID, DLC, raw payload bytes and hardware timestamp) and exposes them as numpy arrays. A thin custom module, `mf4_reader.py` (built on `asammdf` together with numpy and pandas), flattens each frame into a pandas DataFrame with the columns described in Table 3.1; a companion converter (`mf4_converter.py`) uses the same `asammdf` backend to export the raw frames to SavvyCAN-compatible CSV for cross-checking. The physical battery quantities are then obtained by applying the per-DID scaling formulas of the master PID list (Section 3.6) to the decoded UDS response bytes.

Table 3.1.: Normalised frame format used across the analysis pipeline.

Column	Type	Description
timestamp	float (index)	Seconds from recording start.
can_id	int	11-bit CAN arbitration ID.
can_id_hex	string	Hex representation, e.g. 0x065.
dlc	int	Data-length code, 1–8.
data_bytes	byte array	Raw payload.
data_hex	string	Space-separated hex payload.
bus_channel	int	CAN channel (fixed at 9).
direction	int	0 = received.
source	string	Source log folder identifier.

On top of `mf4_reader.py` the analysis pipeline comprises:

- `can_analyzer.py` – enumerates the CAN IDs present in a log and counts the request/response frames, used to verify that each UDS request received an answer.
- `uds_decoder.py` – extracts the UDS response frames that the CANedge2 logged on the BMS reply ID 0x17FE007B, matches each one to its request, and decodes the Data Identifier payloads into physical battery parameters using the per-DID scaling formulas.

3.4. Why Active UDS Instead of Passive Reverse Engineering

Classical CAN reverse engineering is *passive*: the analyst taps the bus on which the target ECU broadcasts, records its frames and recovers the signal layout statistically. This was the originally planned approach for the BMS of the ID. Buzz, but it is not feasible from the OBD-II connector. The BMS broadcasts its raw measurement frames only on the private internal battery/powertrain CAN bus. On the MEB platform a central gateway ECU separates these internal buses from the OBD-II diagnostic bus and, for security reasons, does not forward the raw BMS broadcast frames to the outside. Consequently, the broadcast traffic visible on the OBD-II port does not contain the BMS measurement frames, and a passive byte-level reverse engineering of the battery signals cannot be performed without invasively tapping the high-voltage battery wiring – which is out of scope and unsafe on a production vehicle.

What the gateway *does* forward are addressed UDS diagnostic requests. The BMS answers these on its diagnostic reply ID, so the battery data is reachable actively even though it is not reachable passively. The work therefore proceeds entirely via UDS.

3.5. Reverse-Engineering Methodology

Even though the data is obtained actively, recovering its meaning is still a reverse-engineering task: the manufacturer-specific Data Identifiers are undocumented, and the byte order and scaling of each response must be reconstructed and verified. The methodology applied to every DID is:

Candidate byte layout. For each positive UDS response the payload bytes are interpreted under candidate layouts – single byte, 16-bit big-endian pair, 24/32-bit value, or signed/offset variants – guided by the expected physical range of the parameter.

Scaling factor estimation. Automotive-standard scaling rules ($\text{raw}/2.5$ for SOC in %, $\text{raw}-40$ for temperature in °C, $\text{raw}_{16}/4$ for pack voltage, $\text{raw}_{16}/1000 + 1$ for cell voltage, etc.) are applied and the resulting physical value is checked for plausibility. The correct scaling is the one that maps the raw bytes into the physically expected band.

Cross-signal consistency. Decoded values that must be related are checked against each other – for example, the pack voltage must approximately equal the number of series cells times the mean cell voltage. A scaling that satisfies several such relations simultaneously is taken as confirmed.

Validation against the physical vehicle. The decisive check is the comparison of each decoded value against the real, externally observable state of the test vehicle: the SOC shown on the dashboard, the pack voltage, the ambient temperature, and the operating mode (standby / driving / charging). A DID is only accepted as correctly reverse engineered when its decoded value tracks the corresponding physical reference.

3.6. UDS Diagnostic Procedure for Battery Data

In contrast to earlier plans that foresaw a separate diagnostic tester (a Python host driving a Peak PCAN-USB interface), the active retrieval in this work is carried out entirely by the CANedge2 transmit list. The same device emits the UDS requests and records the BMS responses, so requests and responses share one hardware-timestamped MF4 file. The procedure follows ISO 14229 [5]:

1. The transmit list opens an *extended diagnostic session* on the BMS by sending SID 0x10 sub-function 0x03 to the physical request ID 0x17FC007B.
2. A *TesterPresent* frame (SID 0x3E sub-function 0x00) is sent on every cycle to keep the extended session alive within the S₃ timeout.
3. A list of service 0x22 *ReadDataByIdentifier* requests is transmitted, one per Data Identifier. The candidate identifiers are taken from a master list of all documented VW MEB UDS PIDs (VW MEB UDS PIDs list.csv, 164 entries, compiled for the VW ID.3 and cross-checked against community databases [15]). From this master list the parameters of interest for battery state were flagged for extraction; the resulting selection of **59 Data Identifiers** (Table A.1) deliberately stays below the **64-entry hardware limit** of the CANedge2 transmit list. Each read is a single CAN frame 0x03 22 XX XX 55 55 55 55 (single-frame PCI 0x03, three payload bytes, 0x55 VW padding).
4. Each request is repeated with a 10 s period and the entries are staggered by 150 ms so that the bus has time to drain each response before the next request is sent.
5. Every positive response is decoded offline with the scaling rules of Section 3.5 and validated against the physical reference values of the vehicle (Section 3.4).

The transmit list is provided to the device as a JSON configuration on the SD card. Listing 3.1 shows the two settings that had to be changed from the factory default to make active diagnostics possible, and Listing 3.2 shows the structure of a single transmit entry.

```

1 "can_1": {
2   "phy": {
3     "mode": 0,           // 0 = Normal (was 1 = Restricted/listen-only
                           ): required to transmit

```

```

4   "bit_rate_cfg_mode": 1,      // 1 = manual (was 0 = auto-detect): go bus-
      active immediately
5   "bit_rate_std": 500000,     // Classical CAN arbitration bit rate = 500
      kbit/s (matches the ID. Buzz)
6   "bit_rate_fd": 2000000     // CAN-FD data rate: defined only so the
      controller initialises (CAN-FD unused)
7   }
8   }

```

Listing 3.1: CANedge2 PHY configuration changes required for UDS transmission. The factory default is restricted (listen-only) PHY mode with auto bit-rate detection, which never lets the controller transmit.

```

1   {
2   "name": "T1_SOC",
3   "id": 401997947,           // 0x17FC007B, BMS physical request ID
4   "id_format": 1,           // 1 = 29-bit extended identifier
5   "data_field": "0322028C55555555", // SF PCI 03, SID 22 Read, DID 028C, 0
      x55 padding
6   "period_ms": 10000,       // repeat every 10 s
7   "delay_ms": 450           // stagger within the cycle so responses do
      not collide
8   }

```

Listing 3.2: Structure of a single transmit-list entry: the SOC read (0x22 02 8C) to the BMS at request ID 0x17FC007B, repeated every 10 s.

The full deployment uses several rotated configurations because the transmit list is capped at 64 entries while the battery exposes 108 cells, 18 temperature points and a number of non-BMS ECUs. A `smoketest` configuration (five reads plus `TesterPresent`) first verifies bus, bit rate and extended-ID addressing; a `drive` configuration then covers the system-wide signals during driving, and two complementary `cells-front/cells-rear` configurations cover all 108 individual cell voltages across separate parked/charging sessions. All configurations share an identical five-signal core (SOC 0x028C, HV voltage 0x1E3B, HV current 0x1E3D, main temperature 0x2A0B, operating mode 0x7448) so that logs from different rotations can be time-aligned.

A known limitation of this hardware-based approach is that the CANedge2 transmit list sends *single* CAN frames only; it has no ISO-TP segmentation. DIDs whose response exceeds 7 payload bytes (for example VIN 0xF802 or the battery serial number) would return only their first frame and are therefore deliberately excluded from the sweep. This is the direct practical consequence of the OBD-II bus being Classical CAN rather than CAN-FD.

4. Results

This chapter presents the measurement results of the active UDS reverse engineering. Section 4.1 gives an overview of the UDS logging campaign; Section 4.2 reports the response coverage, the battery parameters decoded from the BMS, and their validation against the physical state of the vehicle.

4.1. UDS Logging Campaign

The active UDS retrieval produced a set of MF4 logs in which the CANedge2 both transmitted the 59 requests and recorded the BMS responses. Table 4.1 summarises the sessions used for the analysis. The longest usable session (00000039, 533 s, 21 438 frames) is taken as the representative log for the decoded values in Section 4.2; the shorter sessions confirm that the response coverage (52/60 positive) is stable across recordings.

Table 4.1.: UDS logging sessions recorded on the VW ID. Buzz (device 2A73E1CC). Frame counts include both the transmitted requests and the logged responses.

Log	Duration [s]	Frames	Positive responses
00000039	533.5	21 438	52 / 60
00000040	333.3	12 630	52 / 60
00000041	233.1	8 526	52 / 60

4.2. Active Battery Data Retrieval via UDS

Following the procedure described in Section 3.6, the CANedge2 transmit list was deployed with the 59 selected Data Identifiers of Table A.1 and the resulting MF4 logs were decoded with `asammdf`. This section reports the values that were actually extracted from the vehicle.

4.2.1. Response Coverage

Across the logging sessions, of the 59 selected identifiers (60 transmit-list reads including the two split charge/discharge fields of 0x1E32), **52 returned a complete positive response**. The remaining identifiers fall into two groups that are entirely explained by the hardware:

- **Six multi-frame identifiers** (0x2AF7, 0x0800, 0xF802 VIN, 0x0500 battery serial, 0x1E32 total charge/discharge, 0x1E3D pack current) returned only their First Frame. As predicted in Section 3.6, the CANedge2 transmit list has no ISO-TP support, so any response longer than 7 payload bytes is truncated.
- **Two identifiers** (0x2AB2, 0x2AB8 energy content) returned no response at all on the OBD-II bus and are therefore not reachable without additional addressing.

4.2.2. Extracted Battery Parameters

Table 4.2 lists the physical values decoded from a representative session (log 00000039, 533 s, 21 438 frames, recorded at $\approx 73\%$ SOC). The values are physically plausible and internally consistent: a pack voltage of 373 V across roughly 96 series cells at 3.89 V per cell ($96 \times 3.89 \approx 373$ V) confirms both the pack- and cell-level scaling factors simultaneously.

Table 4.2.: Battery parameters extracted from the VW ID. Buzz via UDS ReadDataByIdentifier (log 00000039). Values are the decoded physical quantities after applying the per-DID scaling of the master PID list.

Parameter	DID	Scaling	Measured value
State of charge (BMS)	0x028C	raw/2.5 [%]	72.8–73.2 %
Pack voltage	0x1E3B	(raw ₁₆)/4 [V]	373.25–373.75 V
Cell voltage (cells 1–93)	0x1E40+	(raw ₁₆)/1000 + 1 [V]	3.89 V
Cell with highest voltage	0x1E33	cell index	cell 62
Cell with lowest voltage	0x1E34	cell index	cell 62
Battery temperature (main)	0x2A0B	raw/2 – 40 [°C]	16.0 °C
Temperature points 1–18	0x1EAE+	(raw ₁₆)/8 – 40 [°C]	16.0–16.4 °C
PTC heater current	0x1620	raw/4 [A]	0.0 A
Outdoor temperature	0x2609	raw/2 – 50 [°C]	16.5–18.0 °C
Car operation mode	0x7448	enum {0,1,4,6}	0 (standby)

A representative single-frame exchange for the pack voltage (DID 0x1E3B) is:

```

1 >>> 03 22 1E 3B 55 55 55 55      # SF, len=3, SID=22 (Read), DID=1E3B, 0x55
   padding
2 <<< 05 62 1E 3B 05 D5 aa aa      # SF, len=5, SID=62 (pos), payload 0x05D5
3                                     # 0x05D5 = 1493 -> 1493/4 = 373.25 V

```

Listing 4.1: Raw UDS exchange for the pack voltage DID, as logged by the CANedge2.

4.2.3. Anomalies

Three of the sampled cell-voltage identifiers (0x1EA0 cell 97, 0x1EA3 cell 100, 0x1EA7 cell 104) decoded to a constant 5.09 V, which is outside the physically plausible Li-ion range. Because the ID. Buzz pack of this configuration contains 108 cells but the plausible cell readings stop in the mid-90s, these high-index identifiers most likely address reserved or non-populated cell slots whose raw value does not follow the same scaling. They are flagged as anomalies rather than valid measurements. Likewise the *cell index* identifiers 0x1E33/0x1E34 return the cell *number* (62), not a voltage, and must not be interpreted with the cell-voltage scaling.

4.2.4. Validation Against the Physical Vehicle

Because the DIDs are undocumented, the decoded values are only trustworthy once they agree with the real state of the test vehicle. Four independent checks confirm the reverse-engineered scalings:

- **State of charge.** The decoded SOC of 72.8–73.2 % matches the value shown on the vehicle’s own dashboard for the same session (the vehicle had been charged to the low-70 % range before the recording).
- **Pack voltage vs. cell voltages.** The decoded pack voltage (373.25–373.75 V) equals the series-cell count times the mean cell voltage ($96 \times 3.89 \approx 373$ V) to within a fraction of a percent, so the pack- and cell-level scalings validate each other.
- **Temperatures.** The decoded battery and outdoor temperatures (16.0 °C and 16.5–18.0 °C) are consistent with the ambient conditions of the measurement and with each other.
- **Operating mode.** The 0x7448 operating-mode enumeration only ever took the documented values {0,1,4,6}, consistent with the actual standby/charging state of the vehicle during the recordings.

These agreements between the decoded UDS values and the physically observable vehicle state are what justify treating the extraction as a successful reverse engineering of the battery signals, rather than a mere reading of unverified bytes.

5. Discussion

5.1. Comparison with Related Work

The results are qualitatively consistent with the diagnostic reverse-engineering work on other BEVs [18, 19]: pack-level battery parameters are reachable through standardised UDS `ReadDataByIdentifier` requests, whereas the raw broadcast battery bus is isolated by the gateway and detailed internals are protected by security access. The offset-by-40 temperature encoding and the `0x17FC00xx/0x17FE00xx` diagnostic addressing recovered here match the Volkswagen-group conventions documented by the open-source community [15].

5.2. Limitations

Four main limitations of this work should be stated explicitly:

1. **No ISO-TP segmentation.** The decisive practical limitation is that the CANedge2 transmit list emits single CAN frames only. Six of the selected identifiers (VIN `0xF802`, battery serial `0x0500`, total charge/discharge `0x1E32`, pack current `0x1E3D`, and the 12 V and energy DIDs `0x2AF7/0x0800`) return responses longer than 7 bytes and are therefore truncated to their First Frame. A logger or tester with an ISO-TP stack would be required to read these in full.
2. **64-entry transmit cap.** The CANedge2 can hold at most 64 transmit entries, so only 59 parameters could be requested per deployment. Covering all 108 individual cell voltages plus all system signals exceeds this budget and forced a rotation of configurations (`drive`, `cells-front`, `cells-rear`) across separate sessions rather than a single complete read.
3. **Two unreachable DIDs.** The energy-content identifiers `0x2AB2` and `0x2AB8` returned no response on the OBD-II bus, and three high-index cell identifiers (cells 97, 100, 104) decoded to an implausible constant 5.09 V, indicating reserved or non-populated slots. These could not be resolved from the externally accessible bus alone.
4. **Generalisation.** All results are derived from a single vehicle of a single model year. Firmware updates (ID. Software 4.0 vs. 5.0) may change the DID inventory.

5.3. Practical Applications

Even with the limitations above, the DIDs decoded in this thesis are sufficient to build a fleet-level battery monitor for any MEB-platform vehicle: pack SOC, pack voltage, pack current, cell-voltage min/max, cell-temperature min/max and pack SOH cover the parameter space used in typical battery health analytics in the literature [11, 3]. From a safety-engineering point of view, the ability to decode these signals from an external

tester is a prerequisite for CARISSMA's future research on battery diagnostics and post-crash battery state assessment.

6. Conclusion and Outlook

6.1. Summary

This thesis has presented an active, UDS-based reverse engineering of the traction-battery signalling on a Volkswagen ID. Buzz. Because the raw BMS broadcast bus is isolated by the central gateway and is not exposed on the OBD-II connector, a passive byte-level reverse engineering of the battery signals was not feasible; the work therefore recovered the data actively. An extended diagnostic session with the Battery Management Controller was established directly from the CANedge2 data logger, and the UDS service `ReadDataByIdentifier` was used to retrieve the pack-level battery parameters: state of charge, pack voltage, individual cell voltages, the main and per-point cell temperatures, the operating mode and the outdoor temperature. The undocumented Data Identifiers were reverse engineered by reconstructing their byte layout and scaling, and the decoded values were validated against the physical state of the vehicle – the dashboard SOC, the pack voltage, and the ambient temperature – as well as by internal consistency (pack voltage $\approx$ series-cell count $\times$ mean cell voltage).

6.2. Outlook

Four directions for future work are suggested.

First, the data set should be extended with controlled driving and charging scenarios (constant-speed cruise at 50 and 100 km/h, full accelerations, regenerative braking, DC fast charging). Reading the same DIDs across these dynamic states would sharpen the validation of the current- and power-related parameters, whose value barely changed in the mostly static recordings of this work.

Second, the truncated multi-frame DIDs (VIN, battery serial, total charge/discharge, pack current) should be read in full with an ISO-TP-capable tester, and the two unreachable energy-content DIDs should be addressed through the correct ECU, lifting the current 64-entry / single-frame limitations of the CANedge2 transmit list.

Third, the UDS sweep should be extended to the security-access-protected DIDs. A legitimate route through a dealer-level tester could be explored in co-operation with Volkswagen or a certified partner.

Fourth, the results should be integrated into the CARISSMA toolchain for post-crash battery inspection, where a quick read-out of validated pack-level parameters is operationally valuable.

Bibliography

- [1] S. Schaal, "Volkswagen MEB: Die elektroarchitektur von VW im detail," *Automobil-Elektronik*, 2020.
- [2] R. Xiong, J. Cao, Q. Yu, H. He, and F. Sun, "Critical review on the battery state of charge estimation methods for electric vehicles," *IEEE Access*, vol. 6, pp. 1832–1843, 2018.
- [3] J.-W. Park, Y.-J. Lee, and J.-G. Kim, "Data-driven battery state estimation methods for electric vehicle applications," *Energies*, vol. 14, no. 17, p. 5228, 2021.
- [4] International Organization for Standardization, "ISO 11898-1: Road vehicles – controller area network (can) – part 1: Data link layer and physical signalling." International Standard, 2015.
- [5] International Organization for Standardization, "ISO 14229-1: Road vehicles – unified diagnostic services (uds) – part 1: Application layer." International Standard, 2020.
- [6] International Organization for Standardization, "ISO 15765-2: Road vehicles – diagnostic communication over controller area network (docan) – part 2: Transport protocol and network layer services." International Standard, 2016.
- [7] SAE International, "SAE J1962: Diagnostic connector equivalent to ISO 15031-3." SAE Surface Vehicle Standard, 2016.
- [8] W. Zimmermann and R. Schmidgall, *Bussysteme in der Fahrzeugtechnik: Protokolle, Standards und Softwarearchitektur*. Wiesbaden: Springer Vieweg, 5 ed., 2014.
- [9] K. Reif, *Automotive Mechatronics: Automotive Networking, Driving Stability Systems, Electronics*. Wiesbaden: Springer Vieweg, 2014.
- [10] Volkswagen AG, "MEB – der modulare e-antriebs-baukasten: Technischer hintergrund," tech. rep., Volkswagen AG, Press Release, 2019.
- [11] C. Weng, J. Sun, and H. Peng, "A unified open-circuit-voltage model of lithium-ion batteries for state-of-charge estimation and state-of-health monitoring," *Journal of Power Sources*, vol. 258, pp. 228–237, 2014.
- [12] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, "Experimental security analysis of a modern automobile," in *2010 IEEE Symposium on Security and Privacy*, pp. 447–462, IEEE, 2010.
- [13] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, S. Savage, K. Koscher, T. Kohno, F. Roesner, and D. Molnar, "Comprehensive experimental analyses of automotive attack surfaces," *Proceedings of the 20th USENIX Security Symposium*, pp. 447–462, 2011.
- [14] C. Miller and C. Valasek, "Remote exploitation of an unaltered passenger vehicle," *Black Hat USA*, vol. 2015, 2015.

- [15] comma.ai, “opendbc: Democratizing access to vehicle can data.” GitHub repository, github.com/commaai/opendbc, 2024.
- [16] C. Kidder, “SavvyCAN: Can bus reverse engineering and analysis tool.” GitHub repository, github.com/collin80/SavvyCAN, 2024.
- [17] S. Longari, M. Tribastone, M. Tanelli, and S. Zanero, “CANdito: Improving payload-based detection of attacks on controller area networks,” in *2020 IEEE European Symposium on Security and Privacy Workshops*, pp. 598–607, IEEE, 2020.
- [18] T. Huybrechts, W. Joosen, and M. A. Mustafa, “Reverse engineering of CAN bus data from a Tesla Model S,” in *Proceedings of the ACM Workshop on Automotive and Aerial Vehicle Security*, pp. 3–10, ACM, 2018.
- [19] T. Pfeffer and A. Held, “Automotive diagnostic protocols for electric vehicle battery monitoring,” *SAE Int. Journal of Advances and Current Practices in Mobility*, vol. 4, pp. 1102–1113, 2022.
- [20] CSS Electronics, “CANedge2: 2-channel can/lin bus data logger with wifi.” Product Documentation, CSS Electronics, 2024.
- [21] D. Hrisca, “asammdf: Fast python reader and editor for asam mdm measurement files.” GitHub repository, github.com/danielhrisca/asammdf, 2024.

A. Appendix

A.1. UDS Request Builder

The UDS frames used in Section 3.6 are produced from the helper in Listing A.1.

```
1 def build_read_did_request(did: int) -> bytes:
2     """Build an ISO-TP single frame for SID 0x22."""
3     sid = 0x22
4     hi = (did >> 8) & 0xFF
5     lo = did & 0xFF
6     pci = 0x03 # 3 payload bytes
7     frame = bytes([pci, sid, hi, lo]) + b"\x55" * 4 # 0x55 VW padding
8     return frame
9
10 def parse_read_did_response(frame: bytes) -> tuple[int, bytes]:
11     """Return (DID, payload) from a positive Read response."""
12     pci_len = frame[0] & 0x0F
13     assert frame[1] == 0x62, "not a positive response to Read"
14     did = (frame[2] << 8) | frame[3]
15     payload = bytes(frame[4:4 + pci_len - 3])
16     return did, payload
```

Listing A.1: Python helper that builds a single-frame UDS 0x22 ReadDataByIdentifier request and parses the positive response.

A.2. Selected UDS Data Identifiers

The following list contains the 59 Data Identifiers that were flagged for extraction (True) in the master file VW_MEB_UDS_PIDs_list.csv and programmed into the CANedge2 transmit list. The list stays below the 64-entry hardware limit of the logger.

Table A.1.: The 59 UDS Data Identifiers selected for extraction from the VW ID. Buzz (the rows marked True in the master VW_MEB_UDS_PIDs_list.csv). The CANedge2 transmit list is capped at 64 entries, so the selection had to stay below this limit.

Group	Signal	DID	Unit
Technical	Car operation mode	7448	–
Technical	VIN number of car (from engine ECU)	F802	–
Battery	HV Battery cell # with highest voltage	1E33	°C
Battery	HV Battery cell # with lowest voltage	1E34	°C
Battery	HV Battery cell voltage - cell 1	1E40	V
Battery	HV Battery cell voltage - cell 10	1E49	V
Battery	HV Battery cell voltage - cell 100	1EA3	V

Table A.1 – continued

Group	Signal	DID	Unit
Battery	HV Battery cell voltage - cell 104	1EA7	V
Battery	HV Battery cell voltage - cell 13	1E4C	V
Battery	HV Battery cell voltage - cell 17	1E50	V
Battery	HV Battery cell voltage - cell 20	1E53	V
Battery	HV Battery cell voltage - cell 23	1E56	V
Battery	HV Battery cell voltage - cell 27	1E5A	V
Battery	HV Battery cell voltage - cell 30	1E5D	V
Battery	HV Battery cell voltage - cell 33	1E60	V
Battery	HV Battery cell voltage - cell 37	1E64	V
Battery	HV Battery cell voltage - cell 40	1E67	V
Battery	HV Battery cell voltage - cell 43	1E6A	V
Battery	HV Battery cell voltage - cell 47	1E6E	V
Battery	HV Battery cell voltage - cell 50	1E71	V
Battery	HV Battery cell voltage - cell 53	1E74	V
Battery	HV Battery cell voltage - cell 55	1E76	V
Battery	HV Battery cell voltage - cell 57	1E78	V
Battery	HV Battery cell voltage - cell 60	1E7B	V
Battery	HV Battery cell voltage - cell 63	1E7E	V
Battery	HV Battery cell voltage - cell 67	1E82	V
Battery	HV Battery cell voltage - cell 70	1E85	V
Battery	HV Battery cell voltage - cell 73	1E88	V
Battery	HV Battery cell voltage - cell 77	1E8C	V
Battery	HV Battery cell voltage - cell 80	1E8F	V
Battery	HV Battery cell voltage - cell 83	1E92	V
Battery	HV Battery cell voltage - cell 87	1E96	V
Battery	HV Battery cell voltage - cell 90	1E99	V
Battery	HV Battery cell voltage - cell 93	1E9C	V
Battery	HV Battery cell voltage - cell 97	1EA0	V
Battery	HV Battery current	1E3D	A
Battery	HV Battery max temp and temp point	1E0E	°C
Battery	HV Battery min temp and temp point	1E0F	°C
Battery	HV Battery serial	0500	–
Battery	HV Battery temp (main value)	2A0B	°C
Battery	HV Battery temp point 1	1EAE	°C
Battery	HV Battery temp point 12	1EB9	°C
Battery	HV Battery temp point 15	1EBC	°C
Battery	HV Battery temp point 18	7426	°C
Battery	HV Battery temp point 3	1EB0	°C

Table A.1 – continued

Group	Signal	DID	Unit
Battery	HV Battery temp point 6	1EB3	°C
Battery	HV Battery temp point 9	1EB6	°C
Battery	HV Battery voltage	1E3B	V
Battery	HV battery total charge	1E32	kWh
Battery	HV battery total discharge	1E32	kWh
Battery	PTC heater battery current	1620	A
Battery	SOC (BMS)	028C	%
Battery	SOC (HMI)	?	%
Electrical	12V battery voltage	2AF7	V
Electrical	Dynamic limit for charging	?	A
Electrical	Dynamic limit for discharge	?	A
climate	Outdoor temperature	2609	°C
LSD	ODOMETER	295A	km
LSD	Speed	F40D	km/h